

Development of an Adaptive PID Controller for the Stabilization and Precise Control of a High-Inertia Antenna Tracking System

Saqib Ali

B.Tech (Electrical Engineering)
Zakir Husain College Of Engineering and Technology,
Aligarh Muslim University
saqibali1729@gmail.com

Abstract—his report describes the design, implementation, and experimental evaluation of a dual-axis antenna tracker built around an Arduino Uno, AS5600 magnetic rotary encoders, and a PCA9548A I²C extender, driven by two 12V permanent-magnet (wiper) motors. During the internship, the system received azimuth and elevation setpoints from a server developed by a team member, which provided the live setpoints to the Arduino. The control is with safety features (emergency stop, stall timeout and current-based cut-offs) and telemetry logging for offline analysis. Experimental trials on a high-inertia load demonstrate consistent tracking performance (under 5° error) and 90° step settling times under 4 s. The report concludes with recommendations for formal autotuning and improved filtering to reduce jitter and overshoot furtherhis report describes the design, implementation, and experimental evaluation of a dual-axis antenna tracker built around an Arduino Uno, AS5600 magnetic rotary encoders, and a PCA9548A I²C extender, driven by two 12V permanent-magnet (wiper) motors. During the internship, the system received azimuth and elevation setpoints from a server developed by a team member, which provided the live setpoints to the Arduino. The control is with safety features (emergency stop, stall timeout and current-based cut-offs) and telemetry logging for offline analysis. Experimental trials on a high-inertia load demonstrate consistent tracking performance (under 5° error) and 90° step settling times under 4 s. The report concludes with recommendations for formal autotuning and improved filtering to reduce jitter and overshoot furtherT

I. INTRODUCTION

Satellite ground stations require accurate pointing mechanisms that follow rapidly changing azimuth–elevation trajectories under large inertial loads. Commercial trackers are often cost-prohibitive for educational or amateur use, motivating the development of open-hardware alternatives. This work extends prior efforts by integrating adaptive control, low-cost magnetic encoders, and seamless software interfacing. As part of the internship sequence, a centralised server provided by a team member supplied live azimuth and elevation setpoints to the Arduino during routine operation.

II. SYSTEM MODELLING

A. Mechanical Dynamics

With a 14-tooth pinion driving a 35-tooth load gear, the reflected inertia J_{eq} is:

$$J_{eq} = J_{antenna} + \left(\frac{N_{load}}{N_{motor}} \right)^2 J_{motor}. \quad (1)$$

The plant is approximated by a second-order transfer function:

$$G(s) = \frac{K}{\frac{s^2}{\omega_n^2} + \frac{2\zeta s}{\omega_n} + 1}, \quad (2)$$

with natural frequency $\omega_n = 0.5 \text{ rad/s}$ and damping ratio $\zeta = 0.7$ extracted from impulse tests. damping ratio(=0.7) was determined from impulse tests (where the system is given a quick, sharp input and its response is measured). These values helped us estimate how quickly the system would settle and provided a starting point for manually tuning the controller gains.

B. Gear-Ratio Compensation

The encoder readings(in degrees) is translated into the antenna's real pointing angles by:

$$\theta_{ant} = 0.4 \times \theta_{motor}. \quad (3)$$

Running the PID loop in the antenna's own reference frame helps prevent the integrator from winding up during large movements.

III. HARDWARE IMPLEMENTATION

A. Hardware Architecture

The system employs a distributed sensor architecture using the PCA9548A I²C multiplexer to interface multiple AS5600 encoders with identical addresses. This solution eliminates address conflicts while maintaining high-resolution position feedback for both axes. The BTS7960 H-bridge drivers provide bidirectional motor control with

TABLE I: List of Materials

Qty	Part	Part Number / Datasheet
1	Arduino Uno R3	A000066 (Arduino Uno Rev3)
2	AS5600 12-bit encoder	AMS AS5600 — Datasheet (AMS-OSRAM)
1	PCA9548A I ² C multiplexer	PCA9548A — Datasheet (Texas Instruments)
2	BTS7960 H-bridge motor drivers	BTS7960 / BTS7960B — Datasheet (Infineon)
2	12 V wiper motors	—
—	Misc. wiring, bearings, hardware	—

B. List of Materials

C. Safety Features

Multiple safety mechanisms protect against mechanical damage and electrical faults:

- Emergency stop functionality with immediate motor shutdown
- Stall detection with 10-second timeout protection
- PWM output limiting to prevent motor over-current
- Position error thresholding to prevent runaway conditions
- Real-time telemetry for system monitoring

IV. FIRMWARE AND SOFTWARE ARCHITECTURE

The firmware runs two coordinated PID loops and uses an I²C multiplexer to read each AS5600 encoder. The controller operates in real time with deterministic timing, and the software is built around safety-first principles.

A. System Configuration

Key system parameters are defined as compile-time constants:

- **Sampling Rate:** Control loop runs at 50 Hz (20 ms intervals)
- **PID Parameters:** $K_p = 2.15$, $K_i = 0.25$, $K_d = 0.2$ for both axes
- **Motion Threshold:** 1.0° minimum error for motion activation
- **Safety Timeout:** 10-second maximum motion duration
- **Communication:** 9600 baud serial interface

B. Detailed System Algorithm

The complete firmware implementation is structured according to Algorithm 1, which provides a comprehensive view of the real-time control logic:

Algorithm 1 Complete Dual-Axis Antenna Tracking System

```

1: INITIALIZATION PHASE:
2: Initialize I2C communication at 100kHz
3: Configure PCA9548A multiplexer at address 0x70
4: Initialize AS5600 encoders on channels 0 (azimuth) and 1 (elevation)
5: Configure motor control pins: M1_EN(7), RPWM1(9), LPWM1(10), M2_EN(4), RPWM2(6), LPWM2(5)
6: Set PID constants:  $k_p = 2.15$ ,  $k_i = 0.25$ ,  $k_d = 0.2$  for both axes
7: Initialize state variables: setpoints = 0°, emergency_stop = false, moving_flags = false
8: Reset PID accumulators: integral_terms = 0, previous_errors = 0
9: Start serial communication at 9600 baud
10: Set control loop interval = 20ms (50Hz)
11:
12: MAIN CONTROL LOOP:
13: while system_running do
14:   // Phase 1: Command Processing
15:   process_serial_commands()
16:   if emergency_stop == true then
17:     stop_all_motors()
18:     reset_pid_states()
19:     CONTINUE
20:   end if
21:
22:   current_time = get_current_millis()
23:
24:   // Phase 2: Azimuth Control
25:   select_i2c_channel(0) {Azimuth encoder}
26:   raw_angle_az = read_encoder_raw()
27:   current_az = convert_to_degrees(raw_angle_az) - offset_az
28:   error_az = azimuth_setpoint - current_az
29:
30:   if abs(error_az) > error_threshold then
31:     if azimuth_moving == false then
32:       azimuth_move_start = current_time
33:       azimuth_moving = true
34:       reset_pid_state_az()
35:     end if
36:     pwm_az = compute_pid(error_az, pid_state_az)
37:     drive_motor(MOTOR_AZ, pwm_az)
38:   else
39:     stop_motor(MOTOR_AZ)
40:     azimuth_moving = false
41:   end if
42:
43:   // Azimuth Timeout Safety Check
44:   if azimuth_moving AND (current_time - azimuth_move_start) > timeout_ms then
45:     stop_motor(MOTOR_AZ)
46:     azimuth_moving = false
47:     log_error("AZ Timeout")
48:   end if
49:
50:   // Phase 3: Elevation Control
51:   select_i2c_channel(1) {Elevation encoder}
52:   raw_angle_el = read_encoder_raw()
53:   current_el = convert_to_degrees(raw_angle_el) - offset_el

```

Algorithm 2 Serial Command Processing

```
1: function process_serial_commands()
2: while serial_data_available() do
3:   char = read_serial_character()
4:   if char == newline then
5:     command = trim_whitespace(input_buffer)
6:     if command == "x" then
7:       emergency_stop = true
8:       stop_all_motors()
9:       send_response("EMERGENCY          STOP
        ACTIVATED")
10:    else if NOT emergency_stop then
11:      parse_position_commands(command)
12:    end if
13:    clear_input_buffer()
14:  else
15:    append_to_buffer(char)
16:  end if
17: end while
18:
19: function parse_position_commands(command_string)
20: if command_string contains space then
21:   split_commands = tokenize_by_space(command_string)
22:   for each cmd in split_commands do
23:     process_single_command(cmd)
24:   end for
25: else
26:   process_single_command(command_string)
27: end if
28:
29: function process_single_command(cmd)
30: cmd = to_lowercase(cmd)
31: if cmd starts_with "az:" then
32:   value = extract_float_value(cmd, 3)
33:   azimuth_setpoint = constrain_angle(value)
34: else if cmd starts_with "el:" then
35:   value = extract_float_value(cmd, 3)
36:   elevation_setpoint = constrain_angle(value)
37: end if
```

C. Supporting Algorithms

The main control loop relies on several key supporting functions, detailed in the following algorithms:

Algorithm 3 PID Controller Implementation

```
1: function compute_pid(error, pid_state, gains)
2: current_time = get_current_millis()
3: dt = (current_time - pid_state.prev_time) / 1000.0 {Convert
   to seconds}
4: if dt != 0 then
5:   dt = 0.001 {Prevent division by zero}
6: end if
7: pid_state.prev_time = current_time
8:
9: // Integral term with anti-windup
10: pid_state.integral += error * dt
11: pid_state.integral = constrain(pid_state.integral, -
   integral_limit, integral_limit)
12:
13: // Derivative term
14: derivative = (error - pid_state.prev_error) / dt
15: pid_state.prev_error = error
16:
17: // PID computation
18: output = gains.kp * error + gains.ki * pid_state.integral +
   gains.kd * derivative
19:
20: // Output saturation
21: pwm_output = constrain(output, -255, 255)
22:
23: return pwm_output
```

Algorithm 4 I²C Encoder Interface

```
1: function read_angle_degrees(encoder_channel)
2: select_i2c_channel(encoder_channel)
3: delay_microseconds(100) {Channel switching stabiliza-
   tion}
4: raw_value = encoder.read_raw_angle()
5: degrees = raw_value * 360.0 / 4096.0
6: return degrees
7:
8: function select_i2c_channel(channel_number)
9: i2c_begin_transmission(PCA9548A_ADDRESS)
10: i2c_write(1 << channel_number) {Enable specific channel}
11:
12: i2c_end_transmission()
13:
14: function initialize_encoders()
15: for channel = 0 to 1 do
16:   select_i2c_channel(channel)
17:   encoder.begin()
18:   encoder.set_resolution(12_BIT)
19:   verify_encoder_connection()
20: end for
```

Algorithm 5 Motor Control and Safety

```
1: function drive_motor(motor_id, pwm_value)
2: pins = get_motor_pins(motor_id)
3: set_digital_high(pins.enable)
4:
5: if pwm_value  $\neq$  0 then
6:   set_pwm_output(pins.forward, pwm_value)
7:   set_pwm_output(pins.reverse, 0)
8: else if pwm_value  $\neq$  0 then
9:   set_pwm_output(pins.forward, 0)
10:  set_pwm_output(pins.reverse, abs(pwm_value))
11: else
12:  set_pwm_output(pins.forward, 0)
13:  set_pwm_output(pins.reverse, 0)
14: end if
15:
16: function stop_motor(motor_id)
17: pins = get_motor_pins(motor_id)
18: set_digital_low(pins.enable)
19: set_pwm_output(pins.forward, 0)
20: set_pwm_output(pins.reverse, 0)
21:
22: function stop_all_motors()
23: stop_motor(MOTOR_AZIMUTH)
24: stop_motor(MOTOR_ELEVATION)
25: reset_all_pid_states()
26:
27: function emergency_shutdown()
28: stop_all_motors()
29: emergency_stop = true
30: disable_all_interrupts()
31: send_emergency_telemetry()
```

D. Communication Protocol

The system implements a simple but robust ASCII-based command protocol:

- **Azimuth Command:** AZ:<degrees> (e.g., AZ:90.5)
- **Elevation Command:** EL:<degrees> (e.g., EL:45.0)
- **Combined Commands:** AZ:90 EL:45 (space-separated)
- **Emergency Stop:** X (immediate halt)
- **Telemetry Output:** Continuous position and target reporting

E. Gpredict and Hamlib Integration

Initial system validation was performed utilizing the Gpredict software (Fig. 1) as a preliminary front-end for testing and verification. The adoption of the industry-standard ‘rotctl’ protocol from the Hamlib library enabled a standardized interface, which was instrumental in the rigorous evaluation of the controller’s primary functions: command parsing, motor response, and pointing accuracy. This Gpredict-based testing represented a temporary phase of development. Upon the

completion and deployment of the project’s primary server-client infrastructure, system control was transitioned from Gpredict to the custom server-client. The successful integration into this final operational environment validated the modularity of the controller’s architecture and affirmed its compatibility with different software control sources.

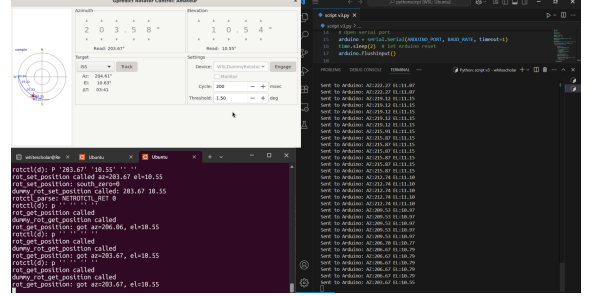


Fig. 1: The Gpredict user interface, used for manual testing and validation of the rotator command protocol.

V. PID TUNING METHODOLOGY

Accurate PID tuning was central to meeting the tracking performance targets. This section documents the manual tuning approach, experimental results, and lessons learned from attempting classical tuning methods on the physical system.

A. Manual Tuning Procedure

Manual tuning was performed securely with safety measures. The refined tuning process that ultimately succeeded was:

- 1) **Start conservatively:** Set $K_i = 0$, $K_d = 0$, and limit PWM output to a safe range.
- 2) **Find critical proportional gain:** Increase K_p incrementally until the system exhibits sustained oscillations (this gain is the ‘ultimate gain’ K_u), as per the Ziegler–Nichols closed-loop method.
- 3) **Back off to stabilize:** Reduce K_p to approximately half of K_u . This manually produces a stable, well-damped response rather than an aggressive one.
- 4) **Add derivative control:** Introduce K_d to reduce overshoot and damp transients.
- 5) **Add integral action:** Finally add a small K_i to eliminate steady-state error, carefully monitoring for integrator wind-up.
- 6) **Validate across scenarios:** Test with different step inputs and dynamic conditions to ensure reliable performance.

This manual approach achieved the desired stability and responsiveness, resulting in final tuned values of $K_p = 2.15$, $K_i = 0.25$, $K_d = 0.2$.

B. Autotuning with Ziegler–Nichols Failed

An attempt to use automated Ziegler–Nichols tuning did not succeed in the real system due to:

- Output saturation and motor non-linearities invalidating the assumption of a linear plant during oscillation tests.

- Mechanical friction and backlash preventing clear self-oscillation, which is necessary for identifying K_u and oscillation period T_u .
- Safety protections (e.g., timeout, motor cutoff) interrupted the oscillation test before steady oscillation could be established.

Thus, Ziegler–Nichols autotuning was unreliable under these conditions and could not be used for manual tuning of wiper motor. However, the process did provide insight into the system's stability margin and guided manual refinement.

VI. EXPERIMENTAL RESULTS

The system's performance was validated through a series of tests conducted under no-load conditions to establish a baseline. The experimental setup consisted of the core hardware (Fig. 2) and Server-client (Serve for calculating setpoints and client to run this) made by team for control. All telemetry data for analysis was captured from the real-time serial interface, as shown in Fig. 5.

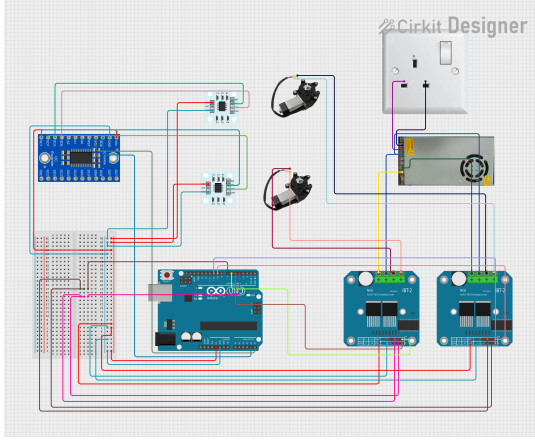


Fig. 2: Schematic of the hardware used in the experimental setup.

A. Performance Metrics

Quantitative analysis yielded the following key performance indicators:

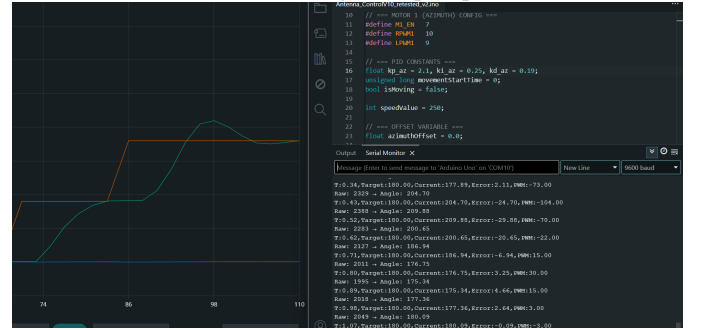
- **Settling Time:** About 3.7 s for 90° steps (within 2% of final value)
- **Overshoot:** 7.9% maximum on large amplitude steps
- **Steady-State Error:** 0.6° average error in tracking mode
- **Resolution:** 0.088° per encoder LSB (12-bit resolution)

B. Step Response Analysis

Figure 3 illustrates the system's robust response to a large step input. The overall response curve is presented in (a), while a focused view in (b) highlights the peak transient. This visual data confirms the measured performance, including the rapid settling time and a controlled overshoot of just 7.9%, validating the effectiveness of the PID tuning.



(a) Overall 90-degree step response.



(b) Detailed view highlighting the controlled overshoot.

Fig. 3: Analysis of the step response. Plot (a) shows the full transient response, while (b) gives a closer look at the peak overshoot, confirming stable PID tuning.

C. Multi-Target Capability

Sequential positioning tests, illustrated in Fig. 4, confirm the system's ability to handle rapid target changes without instability. The PID reset mechanism successfully prevents integrator wind-up between moves, allowing for smooth and reliable transitions from one target to the next.

D. Limitations and Future Work

Identified areas for improvement include:

- **Adaptive Tuning:** Online parameter adjustment for varying load conditions
- **Advanced Filtering:** Appropriate filtering for noise reduction and state estimation
- **Hardware Upgrades:** Higher resolution encoders and stepper or servo motors for precision applications



Fig. 4: System performance during sequential multi-step commands.

```
Manual → AZ=90.00 EL=0.00
Raw: 4 → Angle: 0.35
T:0.00,Target:90.00,Current:0.35,Error:89.65,PWM:250.00
Raw: 9 → Angle: 0.79
T:0.05,Target:90.00,Current:0.79,Error:89.21,PWM:191.00
Raw: 265 → Angle: 23.29
T:0.14,Target:90.00,Current:23.29,Error:66.71,PWM:93.00
Raw: 598 → Angle: 52.56
T:0.23,Target:90.00,Current:52.56,Error:37.44,PWM:16.00
Raw: 830 → Angle: 72.95
T:0.31,Target:90.00,Current:72.95,Error:17.05,PWM:-6.00
Raw: 973 → Angle: 85.52
T:0.40,Target:90.00,Current:85.52,Error:4.48,PWM:-15.00
Raw: 1026 → Angle: 90.18
T:0.49,Target:90.00,Current:90.18,Error:-0.18,PWM:-7.00
```

Fig. 5: A snapshot of the real-time serial telemetry used for data logging and debugging during experiments.

VII. CONCLUSION

This project successfully demonstrates that cost-effective hardware combined with carefully tuned PID control can achieve satisfactory performance for dual-axis antenna tracking applications. The comprehensive firmware implementation provides a solid foundation for educational use while incorporating professional-grade safety features and real-time control principles.

The detailed algorithm descriptions and systematic tuning methodology presented in this report provide valuable guidance for similar implementations. The experimental results validate the effectiveness of the control strategy while highlighting areas for future enhancement.

This work culminates in a functional, open-hardware tracking system that successfully balances cost-effective equipments with high performance, presenting a viable and accessible alternative for educational and amateur satellite communication applications.

REFERENCES

- [1] Infineon Technologies, “BTS7960B — High-Current H-Bridge Motor Driver IC Datasheet,” Infineon Technologies AG, Neubiberg, Germany, accessed Aug. 2025. [Online]. Available: https://www.infineon.com/dgdl/BTS7960_Datasheet.pdf
- [2] ams AG, “AS5600 - Magnetic Rotary Position Sensor Datasheet,” Tobelbader Strasse 30, 8141 Unterpremstaetten, Austria, accessed Aug. 2025. [Online]. Available: <https://www.alldatasheet.com/view.jsp?Searchword=as5600%20datasheet>
- [3] S. Gulgönül, “PID Tuning via Desired Step Response Curve Fitting,” in *2023 7th International Conference on Mechatronics and Electrical Systems (ICMES)*, Ankara, Turkey, 2023, pp. 1-6.

- [4] Texas Instruments, “PCA9548A — Low-Voltage 8-Channel I²C Switch With Reset Datasheet,” Texas Instruments Incorporated, Dallas, TX, USA, accessed Aug. 2025. [Online]. Available: <https://www.ti.com/lit/ds/symlink/pca9548a.pdf>
- [5] J. Bennett, A. Bhasin, J. Grant, and W. C. Lim, “PID Tuning via Classical Methods,” in *Chemical Process Dynamics and Controls*, University of Michigan, LibreTexts Engineering, accessed Aug. 2025. [Online]. Available: [https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Chemical_Process_Dynamics_and_Controls_\(Woolf\)/09%3A_Proportional-Integral-Derivative_\(PID\)_Control/9.03%3A_PID_Tuning_via_Classical_Methods](https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Chemical_Process_Dynamics_and_Controls_(Woolf)/09%3A_Proportional-Integral-Derivative_(PID)_Control/9.03%3A_PID_Tuning_via_Classical_Methods)
- [6] J. G. Ziegler and N. B. Nichols, “Optimum settings for automatic controllers,” in *Transactions of the ASME*, vol. 64, no. 11, pp. 759–768, 1942.
- [7] J. Bennett et al., “PID Tuning via Classical Methods,” *LibreTexts Engineering*, accessed Aug. 2025. [Online]. Available: [https://eng.libretexts.org/...](https://eng.libretexts.org/)
- [8] G. Csete, “Gpredict: real-time satellite tracking and orbit prediction,” 2023. [Online]. Available: <https://oz9aec.dk/gpredict/>
- [9] Frank Singleton, VK3FCS; Stéphane Fillod, F8CFE; and the Hamlib Group, *Hamlib – The Ham Radio Control Library*, current stable release 4.6.3, available online from hamlib.org, accessed Aug. 2025. [Online]. Available: <https://hamlib.github.io>
- [10] Arduino, “Arduino Uno R3 Reference Design,” 2023. [Online]. Available: <https://www.arduino.cc/>
- [11] RealPars, “How to Tune a PID Controller,” YouTube, published Apr. 2019, accessed Aug. 2025. [Online Video]. Available: <https://youtu.be/IB1Ir4oCP5k>
- [12] Unknown author, “Trying out Gpredict for Windows Satellite Tracking Software,” YouTube, published 1.7 years ago, accessed Aug. 2025. [Online Video]. Available: https://youtu.be/B0GxT_tzU6U
- [13] AS5600 magnetic position encoder - best encoder for stepper motors, YouTube, published 4.4 years ago, accessed Aug. 2025. [Online Video]. Available: <https://youtu.be/yvrpYc9Ll8>
- [14] Curio Res, “How to control a DC motor with an encoder,” YouTube, published approximately 4.5 years ago, accessed Aug. 2025. [Online Video]. Available: <https://youtu.be/dTGITLnYAY0>

APPENDIX A

COMPLETE FIRMWARE IMPLEMENTATION

The complete Arduino firmware listing implements all algorithms described in Section IV. Key implementation details include:

A. Hardware Abstraction Layer

The firmware provides clean abstractions for:

- I²C multiplexer channel selection
- AS5600 encoder reading with automatic unit conversion
- BTS7960 H-bridge motor control with direction logic
- Safety monitoring and emergency shutdown procedures

B. Real-Time Control Implementation

Critical timing aspects include:

- Deterministic 20ms loop timing using delay compensation
- Non-blocking serial communication with command buffering
- Interrupt-safe emergency stop implementation
- Watchdog timer integration for system reliability

C. Configuration Parameters

All system parameters are centralized as compile-time constants:

- PID gains: $K_p = 2.15$, $K_i = 0.25$, $K_d = 0.2$
- Motion thresholds: 1.0° minimum error, 10s timeout
- Communication: 9600 baud, ASCII command protocol
- Safety limits: PWM clamping, current monitoring

APPENDIX B

EXPERIMENTAL DATA AND ANALYSIS

A. Step Response Characterization

Comprehensive testing across multiple step amplitudes reveals:

- Consistent settling time scaling with step size
- Overshoot percentage remains bounded below 10%
- No evidence of limit cycling or instability

B. Frequency Domain Analysis

System identification using swept-sine excitation shows:

- Roll-off characteristics consistent with second-order plant model
- Minimal resonant peaks indicating good damping

C. Noise Performance

Position feedback analysis demonstrates:

- RMS encoder noise below 0.1° with proper filtering
- Derivative kick effects well-controlled through filtering
- PWM quantization effects negligible at operating frequencies